

Programação Orientada a Objetos

DAO, CRUD e Tratamento de Exceções

[Baixe aqui os exemplos da aula](#)



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Da Conexão ao CRUD de Verdade

A Aula 06 provou que o canal com o MySQL abre e fecha corretamente, mas nenhum Dao usava essa conexão — DaoBase continuava guardando os clientes numa lista em memória. Esta aula fecha essa lacuna: ClienteDao passa a ler e escrever de verdade na tabela clientes, e o main do GestorOS, sem mudar de interface, começa a persistir dados que sobrevivem ao fechamento do programa.

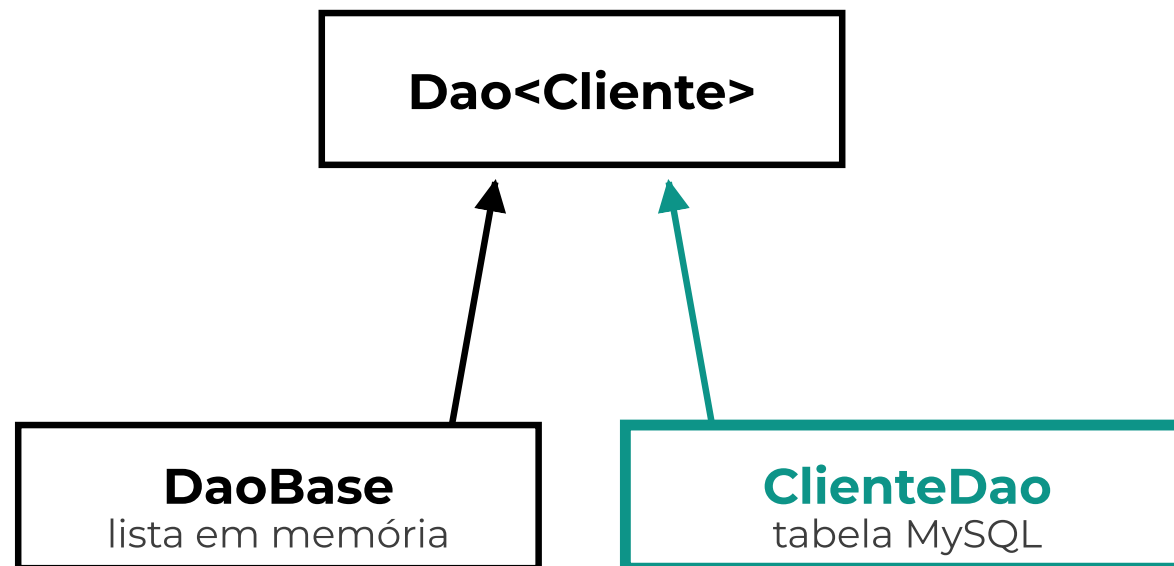


Por Que ClienteDao Deixa de Estender DaoBase

DaoBase resolve salvar, buscar, listar e remover assumindo uma lista em memória — id sequencial, ArrayList, nenhuma chance de falha externa. Uma tabela MySQL não se comporta assim: cada operação abre uma conexão, monta um comando SQL e pode falhar por motivos que a lista nunca teve. Reaproveitar o CRUD de DaoBase faria pouco sentido; o que continua sendo reaproveitado é só o contrato Dao<Cliente>.



Mesma Interface, Duas Representações



O contrato não mudou. A representação por baixo, sim — de novo, o mesmo princípio do TAD.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

PreparedStatement e a Injeção de SQL

Montar um comando concatenando texto recebido de fora ("... WHERE nome = '" + nome + "'") abre a porta para injeção de SQL: um valor malicioso pode alterar o comando inteiro. PreparedStatement separa o comando fixo dos valores, marcados com "?" e enviados depois via `setString/setInt` — o driver garante que cada valor é tratado como dado, nunca como parte do comando.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

ClienteDao.salvar(): INSERT com PreparedStatement

```
String sql = "INSERT INTO clientes (nome, telefone) VALUES (?, ?)";
try (Connection conexao = ConexaoMySQL.obter();
    PreparedStatement comando = conexao.prepareStatement(
        sql, Statement.RETURN_GENERATED_KEYS)) {
    comando.setString(1, objeto.getNome());
    comando.setString(2, objeto.getTelefone());
    comando.executeUpdate();
    // ... lê a chave gerada e chama objeto.setId(...)
    return objeto;
} catch (SQLException e) {
    throw new PersistenciaException(
        "Não foi possível salvar o cliente " + objeto.getNome(), e);
}
```



ClienteDao.listarTodos(): SELECT Reconstrói Objetos

```
String sql = "SELECT id, nome, telefone FROM clientes ORDER BY id";
List<Cliente> clientes = new ArrayList<>();
try (Connection conexao = ConexaoMySQL.obter();
    PreparedStatement comando = conexao.prepareStatement(sql);
    ResultSet resultado = comando.executeQuery()) {
    while (resultado.next()) {
        clientes.add(construir(resultado));
    }
    return clientes;
} catch (SQLException e) {
    throw new PersistenciaException("Não foi possível listar os clientes", e);
}
```



SQLException Não Pode Vazar Sem Tradução

Se ClienteDao deixasse SQLException escapar sem tratamento, qualquer código que chamasse Dao<Cliente> precisaria importar java.sql e saber que, por baixo do contrato, existe um banco relacional — exatamente o acoplamento que a interface deveria evitar. Capturar a exceção técnica e relançar como PersistenciaException devolve ao chamador só o que importa: a operação falhou, e por quê.



De SQLException a um Catch Único



A tradução acontece uma vez, dentro do Dao. O main nunca vê SQLException.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

PersistenciaException: a Exceção de Negócio

```
// Não verificada: a assinatura de Dao<T> não declara throws,  
// e não deveria - uma implementação em memória nunca falharia  
// por rede, e não pode ser obrigada a lidar com essa exceção.  
public class PersistenciaException extends RuntimeException {  
  
    public PersistenciaException(String mensagem, Throwable causa) {  
        super(mensagem, causa);  
    }  
}
```



RuntimeException, Não Checked

Dao<T> declara salvar, buscarPorId, listarTodos e remover sem nenhum throws — e continua assim. Se PersistenciaException fosse verificada, a interface precisaria declarar throws PersistenciaException, obrigando toda implementação futura — mesmo uma em memória, que nunca falha por rede — a lidar com uma exceção que não faz sentido para ela. RuntimeException mantém o contrato limpo.



Onde Capturar de Verdade

Capturar `PersistenciaException` depois de cada chamada a `clienteDao` poluiria o `main` com o mesmo tratamento repetido quatro vezes. A exceção sobe até o ponto em que uma decisão de fato pode ser tomada — aqui, um único bloco `try` no início do `main`, que encerra o programa com uma mensagem clara se o banco estiver inacessível, em vez de uma pilha de exceção despejada no console.



O Main Captura Uma Única Vez

```
try {  
    Cliente marina = clienteDao.salvar(new Cliente(...));  
    Cliente bruno = clienteDao.salvar(new Cliente(...));  
    // ... listarTodos, buscarPorId, remover ...  
    System.out.println("Clientes cadastrados:");  
    for (Cliente cliente : clienteDao.listarTodos()) {  
        System.out.println("    #" + cliente.getId() + " " + cliente);  
    }  
} catch (PersistenciaException e) {  
    System.out.println("Nao foi possivel falar com o banco: "  
        + e.getMessage());  
    return;  
}
```



Atividades

Exercício 1

Por que ClienteDao deixou de estender DaoBase, se as duas classes ainda implementam Dao<Cliente>?

Exercício 2

O que PreparedStatement evita que uma concatenação de String com dados do usuário não evitaria?

Exercício 3

Por que PersistenciaException é uma RuntimeException, e não uma exceção verificada (checked)?



Desafio extra

Opcional

Quem quiser ir além do combinado nesta semana:

1. Implementar um método atualizar(Cliente objeto) em ClienteDao (UPDATE), somando-o à interface Dao<T>.
2. Criar uma PersistenciaException mais específica (ex.: ClienteNaoEncontradoException) para quando buscarPorId não encontra nada, hoje simplesmente devolvendo null.

Nada disso é cobrado na entrega. O projeto segue completo sem essas adições.

